

Name :- Rashmeet Kaur Nanade
Cohort :- Demis Hassabis
Roll No :- 150096725165
Subject :- Backend Development –
Node.js, MongoDB & Firebase.
Course :- B.Tech Computer Science &
Engineering
Professor :- Ricky Kumar

ASSIGNMENT - 1

BASIC HTTP SERVER

Objective :-

Create a basic HTTP server using Node.js that displays a welcome message in the browser.

Problem Statement :-

Develop a Node.js application that :-

- 1. Create an HTTP server using the built-in http module.**
- 2. Run the server on Port 3000.**
- 3. Display the message "Welcome to Node.js Server" in the browser.**
- 4. Log the message "Server is running on http://localhost:3000" in the terminal when the server starts.**

SOURCE CODE

```
JS basicServer.js > ...
1  const http = require('http');
2
3  const server = http.createServer((req, res) => {
4
5      res.end('Welcome to Node.js Server');
6
7  });
8
9  server.listen(3000, () => {
10
11      console.log('Server is running on http://localhost:3000');
12
13  });
14
```

Line 1 – Import HTTP module

```
const http = require('http');
```

This imports Node.js's built-in http module, which is used to create the HTTP server.

Line 2-7- Create the Server

```
const server = http.createServer((req, res) => {
```

The `http.createServer()` method creates an HTTP server.

- `req` represents the request received from the browser.
- `res` represents the response sent back to the browser.

Send response to browser

```
res.end('Welcome to Node.js Server');
```

This sends the required message :-

```
Welcome to Node.js Server
```

to the browser as the server's response.

Start the server on Port 3000

```
server.listen(3000, () => {
```

The listen() method starts the server and makes it listen on Port 3000.

Therefore, the server can be accessed at :-

```
http://localhost:3000
```

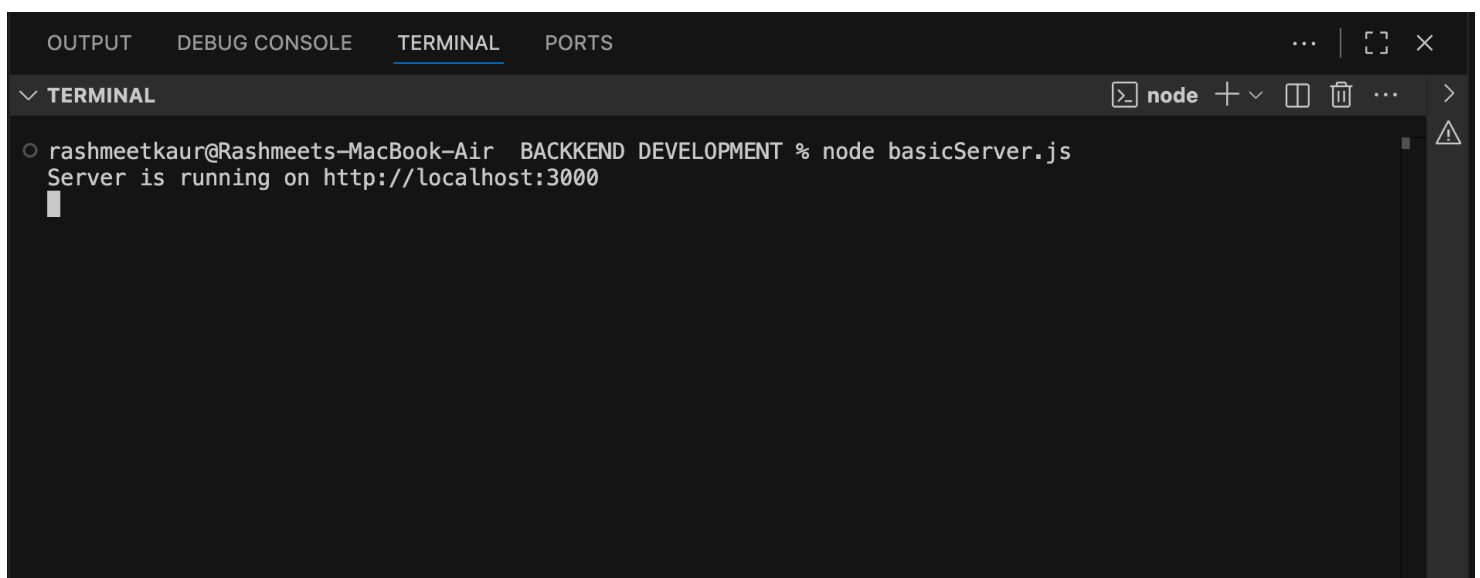
Display message in Terminal

```
console.log('Server is running on http://localhost:3000');
```

This displays a message in the Terminal when the server successfully starts.

TERMINAL OUTPUT

The Node.js server was started using the node basicServer.js command in the VS Code Terminal.

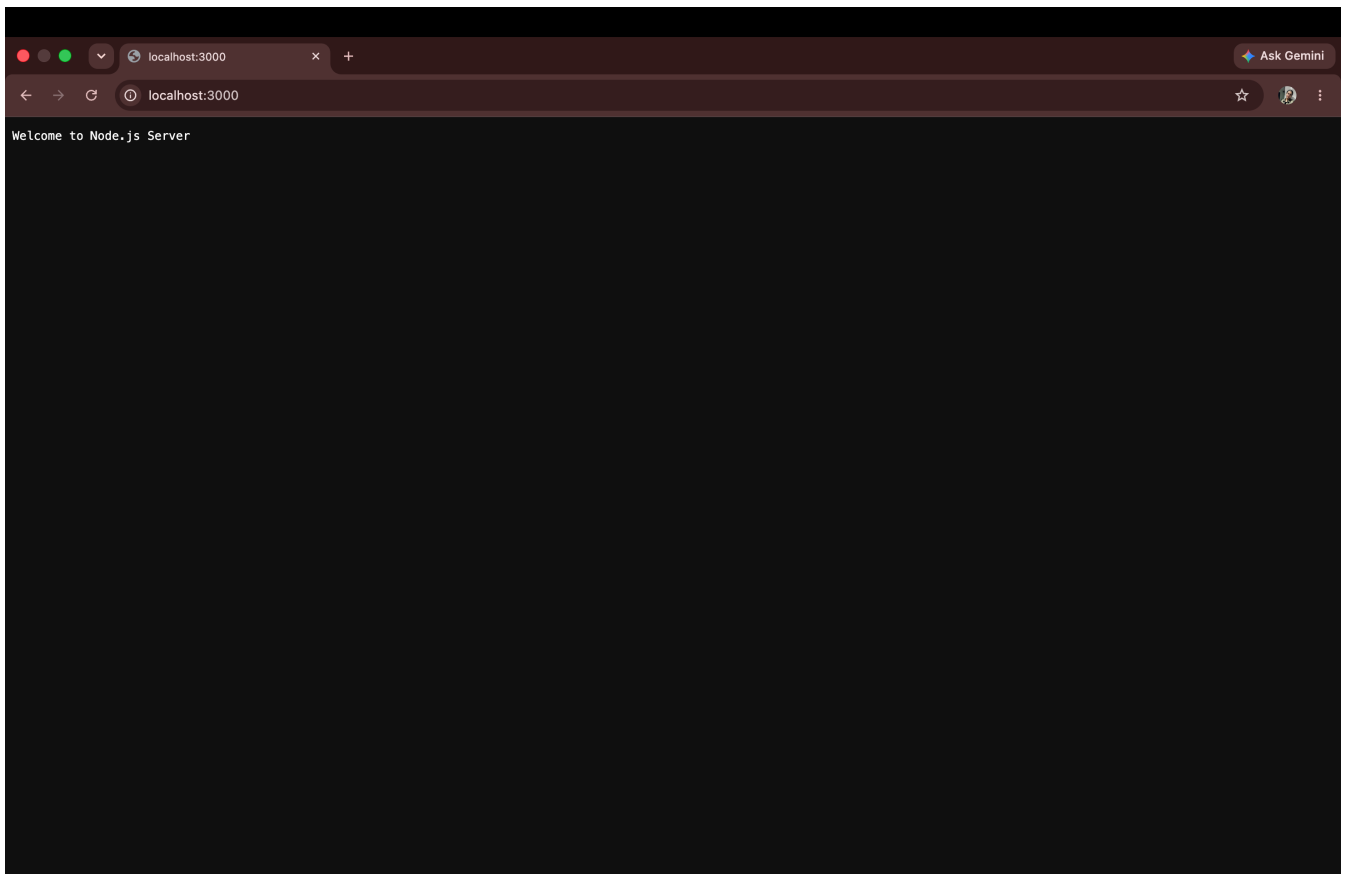


Terminal Output :-

Server is running on <http://localhost:3000>

BROWSER OUTPUT

I opened the server in a browser, and the required message appeared.



Browser Output :-

Welcome to Node.js Server

CONCEPTS COVERED :-

1. `http.createServer()`

Used to create the HTTP server.

2. Server Listening

`server.listen(3000)` starts the server on Port 3000.

3. Request and Response

`req` represents the browser request, while `res` is used to send the response back to the browser.

Name :- Rashmeet Kaur Nanade

Cohort :- Demis Hassabis

Roll No :- 150096725165

**Subject :- Backend Development –
Node.js, MongoDB & Firebase.**

**Course :- B.Tech Computer Science &
Engineering**

Professor :- Ricky Kumar

ASSIGNMENT - 2

HTML RESPONSE SERVER

Objective Serve an HTML page using the Node.js HTTP module.

Problem Statement Create a Node.js server that returns an HTML page containing :-

- A heading: Student Portal
- Student Name
- Course Name
- College Name
- A welcome paragraph The page should display properly in the browser.

SOURCE CODE

```
JS htmlServer.js > ...
1  const http = require('http');
2
3  const server = http.createServer((req, res) => {
4
5      res.writeHead(200, { 'Content-Type': 'text/html' });
6
7      res.end(`
8          <!DOCTYPE html>
9          <html>
10             <head>
11                 <title>Student Portal</title>
12             </head>
13
14             <body>
15
16                 <h1>Student Portal</h1>
17
18                 <p>Name: John Doe</p>
19
20                 <p>Course: Full Stack Development</p>
21
22                 <p>College: XYZ College</p>
23
24                 <p>Welcome to our Node.js application.</p>
25
26             </body>
27             </html>
28         `);
29
30     });
31
32     server.listen(3000, () => {
33
34         console.log('HTML Server is running on http://localhost:3000');
35
36     });
```

1. Importing the HTTP Module

```
const http = require('http');
```

This imports Node.js's built-in http module, which is used to create an HTTP server.

2. Creating the HTTP Server

```
const server = http.createServer((req, res) => {
```

`http.createServer()` creates the server.

Here :-

- `req` represents the request received from the browser.
- `res` represents the response that the server sends back to the browser.

3. Setting the Response Header

```
res.writeHead(200, { 'Content-Type': 'text/html' });
```

This sets the response header.

- 200 means the request was successful.
- `'Content-Type': 'text/html'` tells the browser that the server is sending HTML content.

This is one of the main concepts required in this assignment:
Response Headers.

4. Sending the HTML Content

```
res.end(  
  <!DOCTYPE html>  
  <html>  
  ...  
);
```

`res.end()` sends the HTML content to the browser and ends the response.

The HTML contains :-

```
<h1>Student Portal</h1>
```

which creates the heading.

The `<p>` elements display :-

```
Name: John Doe  
Course: Full Stack Development  
College: XYZ College  
Welcome to our Node.js application.
```

5. Starting the Server

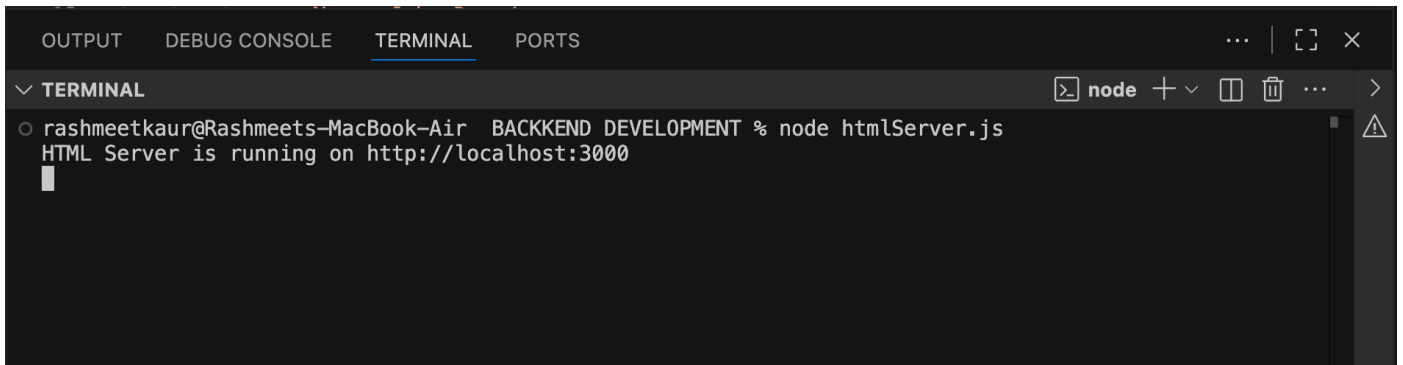
```
server.listen(3000, () => {
```

This starts the Node.js server on Port 3000.

When the server starts, the following message is displayed in the Terminal :-

```
console.log('HTML Server is running on http://localhost:3000');
```

TERMINAL OUTPUT

A screenshot of the Visual Studio Code (VS Code) interface, specifically the TERMINAL panel. The terminal shows a command prompt where the user has run 'node htmlServer.js'. The output of the command is 'HTML Server is running on http://localhost:3000'. The terminal title bar indicates the current session is named 'node'.

```
OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
▼ TERMINAL  node + - [ ] [x] ... >
rashmeetkaur@Rashmeets-MacBook-Air BACKKEND DEVELOPMENT % node htmlServer.js
HTML Server is running on http://localhost:3000
```

The Node.js HTML server was started using the node htmlServer.js command in the VS Code Terminal.

BROWSER OUTPUT

The server was accessed in the browser using http://localhost:3000. The HTML page was successfully displayed in the browser.



CONCEPTS COVERED

1. HTTP Response

The server uses the `res` object to send a response back to the browser. The `res.end()` method sends the HTML content and completes the response.

2. HTML Content

HTML elements such as `<h1>` and `<p>` are sent by the Node.js server and displayed as a webpage in the browser.

3. Response Headers

The following response header :-

```
res.writeHead(200, { 'Content-Type': 'text/html' });
```

sets the status code to 200 and tells the browser that the response contains HTML content.

Name :- Rashmeet Kaur Nanade
Cohort :- Demis Hassabis
Roll No :- 150096725165
**Subject :- Backend Development –
Node.js, MongoDB & Firebase.**
**Course :- B.Tech Computer Science &
Engineering**
Professor :- Ricky Kumar

ASSIGNMENT - 3

STUDENT JSON API

Objective

Create a simple REST-like endpoint that returns JSON data.

Problem Statement

Create a Node.js server with the following behavior :-

- When the user visits /student, return the following JSON object:
- {
- "id": 101,
- "name": "John",
- "course": "BCA",
- "semester": 4, "
- city": "Mumbai" }
- If the user visits any other route, display :- 404 - Page Not Found

SOURCE CODE

```
JS studentAPI.js > ...
1  const http = require('http');
2
3  const server = http.createServer((req, res) => {
4
5      if (req.url === '/student') {
6
7          res.writeHead(200, { 'Content-Type': 'application/json' });
8
9          const student = {
10              id: 101,
11              name: "John",
12              course: "BCA",
13              semester: 4,
14              city: "Mumbai"
15          };
16
17          res.end(JSON.stringify(student));
18
19      } else {
20
21          res.writeHead(404, { 'Content-Type': 'text/plain' });
22
23          res.end('404 - Page Not Found');
24
25      }
26
27  });
28
29  server.listen(3000, () => {
30
31      console.log('Student API server is running on http://localhost:3000');
32
33  });
```

1. HTTP Module

```
const http = require('http');
```

Imports Node.js's built-in HTTP module to create the server.

2. Routing

```
if (req.url === '/student')
```

Checks whether the user has visited the /student route.

3. JSON Response

```
res.writeHead(200, { 'Content-Type': 'application/json' });
```

Sends a successful 200 status and tells the browser that the response is in JSON format.

4. Student Data

The student details are stored in a JavaScript object containing the ID, name, course, semester, and city.

5. Sending JSON

```
res.end(JSON.stringify(student));
```

Converts the student object into JSON and sends it to the browser.

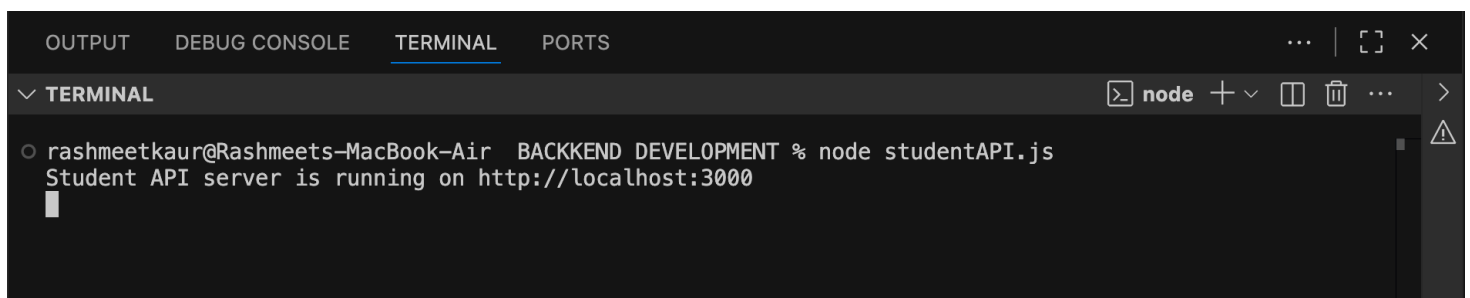
6. 404 Error

```
res.writeHead(404, { 'Content-Type': 'text/plain' });  
res.end('404 - Page Not Found');
```

If the user visits any route other than /student, the server returns a 404 Page Not Found message.

7. Terminal Output

The Student JSON API server was started using the node studentAPI.js command in the VS Code Terminal. The server is running on Port 3000.

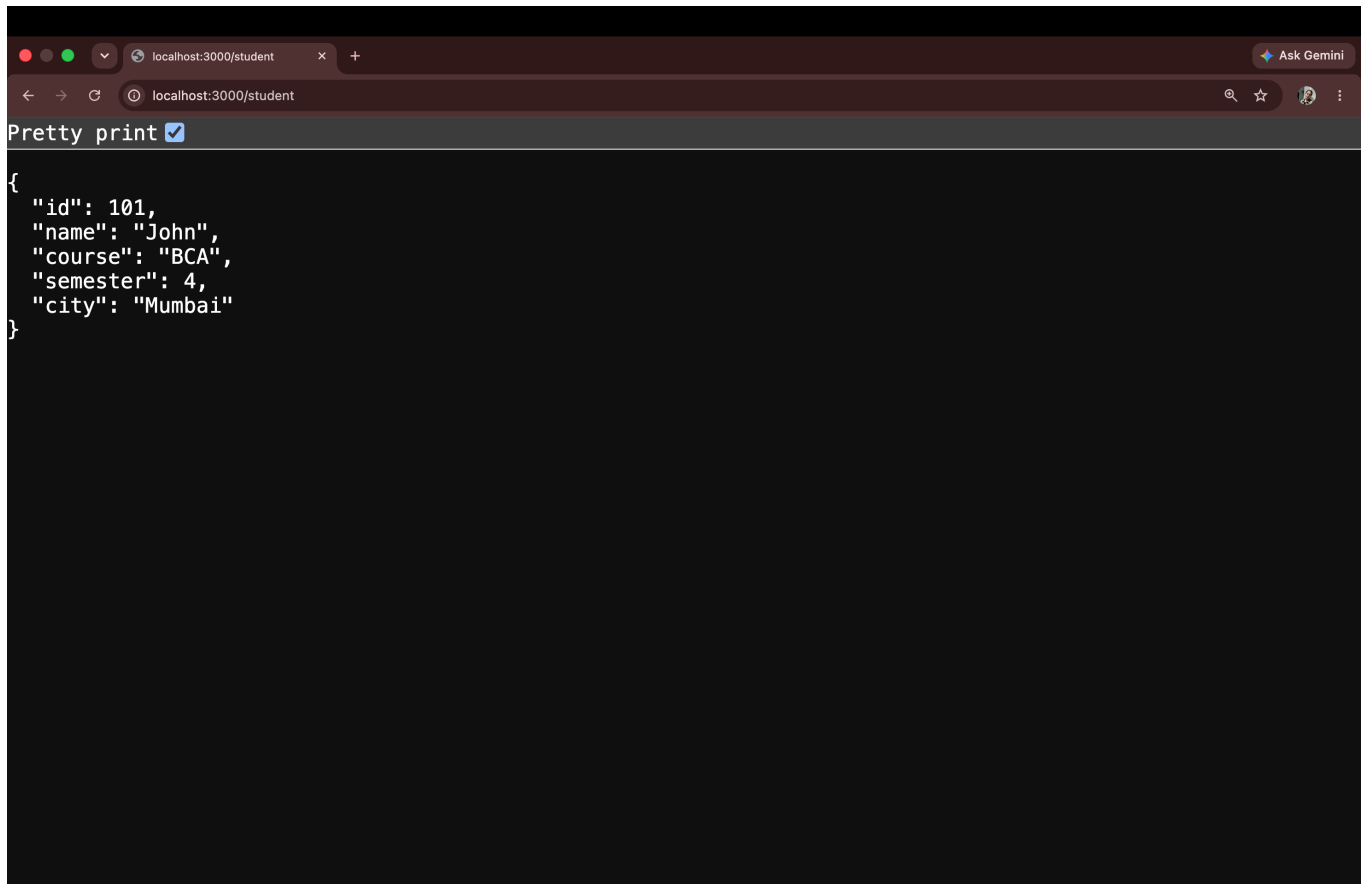


```
OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  
▼ TERMINAL  node + ▾ □ 🗑️ ⋮ >  
rashmeetkaur@Rashmeets-MacBook-Air BACKKEND DEVELOPMENT % node studentAPI.js  
Student API server is running on http://localhost:3000
```

8. Browser Output

8.1 Successful /student Route

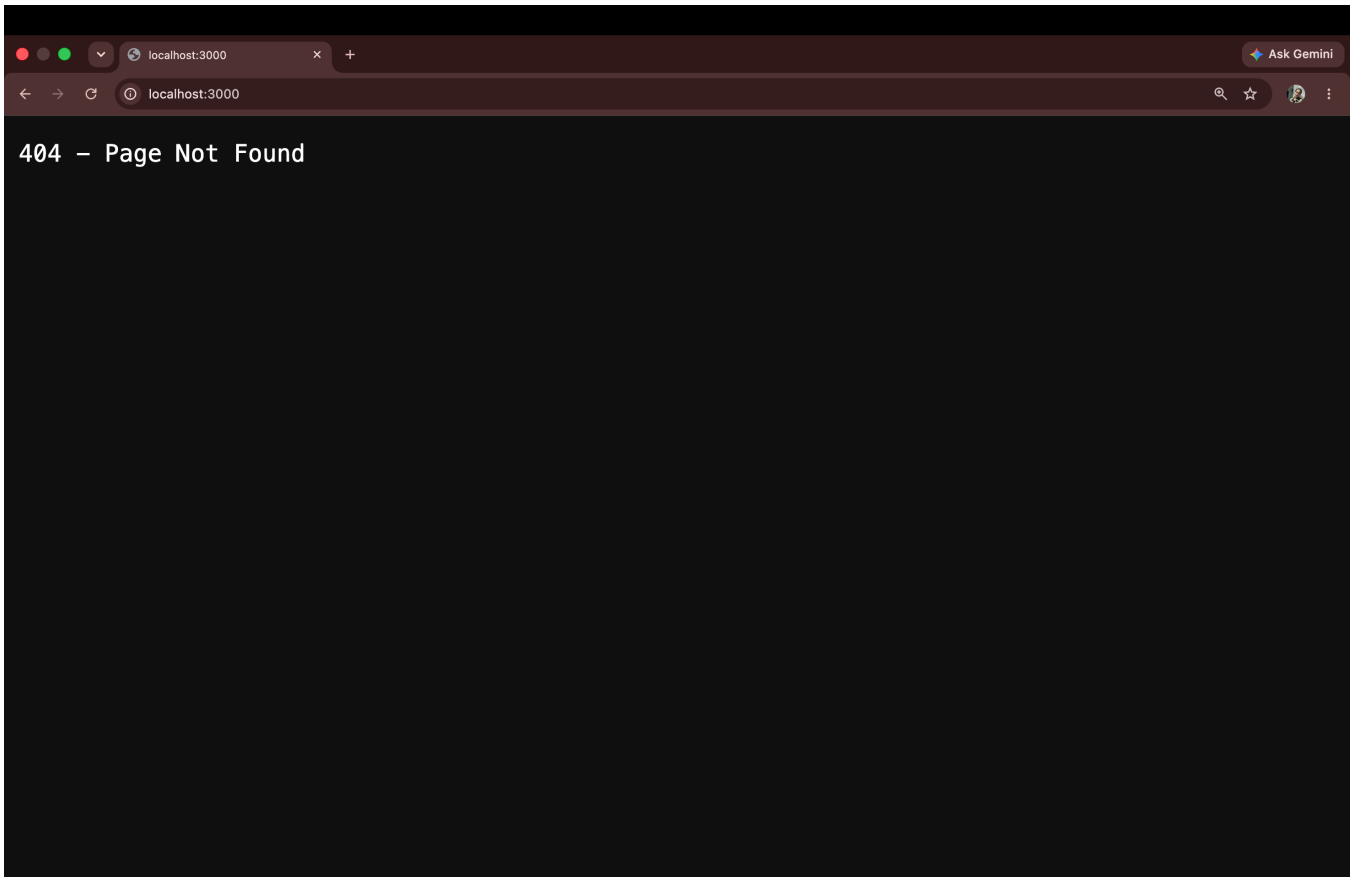
When the user visits /student, the server successfully returns the student information in JSON format.

A screenshot of a web browser window. The address bar shows 'localhost:3000/student'. The page content is a JSON object displayed in a 'Pretty print' format. The JSON object contains the following data: { "id": 101, "name": "John", "course": "BCA", "semester": 4, "city": "Mumbai" }.

```
{
  "id": 101,
  "name": "John",
  "course": "BCA",
  "semester": 4,
  "city": "Mumbai"
}
```

8.2 Invalid Route — 404 Error

When the user visits a route other than /student, the server returns a 404 Page Not Found message.



Expected Browser Output

404 - Page Not Found

Concepts Covered

1. JSON Response

The server returns the student details in JSON format using `JSON.stringify()` and the `application/json` response header.

2. Routing

The server checks the requested URL using `req.url`. If the user visits `/student`, the student JSON data is returned. Other routes return a 404 message.

3. HTTP Status Codes

The server uses 200 for a successful /student request and 404 when the requested route is not found.

Name :- Rashmeet Kaur Nanade
Cohort :- Demis Hassabis
Roll No :- 150096725165
Subject :- Backend Development –
Node.js, MongoDB & Firebase.
Course :- B.Tech Computer Science &
Engineering
Professor :- Ricky Kumar

ASSIGNMENT - 4
ROUTE HANDLING SERVER

SOURCE CODE

```
JS routeServer.js > ...
1  const http = require('http');
2
3  const server = http.createServer((req, res) => {
4
5      if (req.url === '/') {
6          res.writeHead(200, { 'Content-Type': 'text/plain' });
7          res.end('Welcome to Home Page');
8
9      } else if (req.url === '/about') {
10         res.writeHead(200, { 'Content-Type': 'text/plain' });
11         res.end('About Us');
12
13     } else if (req.url === '/contact') {
14         res.writeHead(200, { 'Content-Type': 'text/plain' });
15         res.end('Contact Information');
16
17     } else if (req.url === '/services') {
18         res.writeHead(200, { 'Content-Type': 'text/plain' });
19         res.end('Our Services');
20
21     } else {
22         res.writeHead(404, { 'Content-Type': 'text/plain' });
23         res.end('404 - Page Not Found');
24     }
25
26 });
27
28 server.listen(3000, () => {
29     console.log('Server is running on http://localhost:3000');
30 });
```

1. Importing the HTTP module

```
const http = require('http');
```

The built-in http module is imported to create the Node.js web server.

2. Creating the server

```
const server = http.createServer((req, res) => {
```

`http.createServer()` creates the server.

- `req` represents the request received from the browser.
- `res` represents the response that the server sends back.

3. Home route

```
if (req.url === '/') {
```

This checks whether the user has visited the home page :-

```
http://localhost:3000/
```

The server responds with :-

```
Welcome to Home Page
```

4. About route

```
else if (req.url === '/about') {
```

This handles :-

```
http://localhost:3000/about
```

and displays :-

About Us

5. Contact route

```
else if (req.url === '/contact') {
```

This handles :-

<http://localhost:3000/contact>

and displays :-

Contact Information

6. Services route

```
else if (req.url === '/services') {
```

This handles :-

<http://localhost:3000/services>

and displays :-

Our Services

7. Handling an unknown route

```
else {  
  res.writeHead(404, { 'Content-Type': 'text/plain' });  
  res.end('404 - Page Not Found');  
}
```

If the user enters a route that is not /, /about, /contact, or /services, the server returns :-

```
404 - Page Not Found
```

The 404 status code means that the requested page was not found.

8. Starting the server

```
server.listen(3000, () => {  
  console.log('Server is running on http://localhost:3000');  
});
```

This starts the server on port 3000.

When the server starts successfully, the terminal displays :-

```
Server is running on http://localhost:3000
```


TERMINAL OUTPUT



```
OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  ... | [ ] X
▼ TERMINAL  > node + ▾ [ ] [ ] ... >
○ rashmeetkaur@Rashmeets-MacBook-Air  BACKKEND DEVELOPMENT % node routeServer.js
Server is running on http://localhost:3000
█
```

The terminal confirms that the Node.js server is successfully running on port 3000.

BROWSER OUTPUT

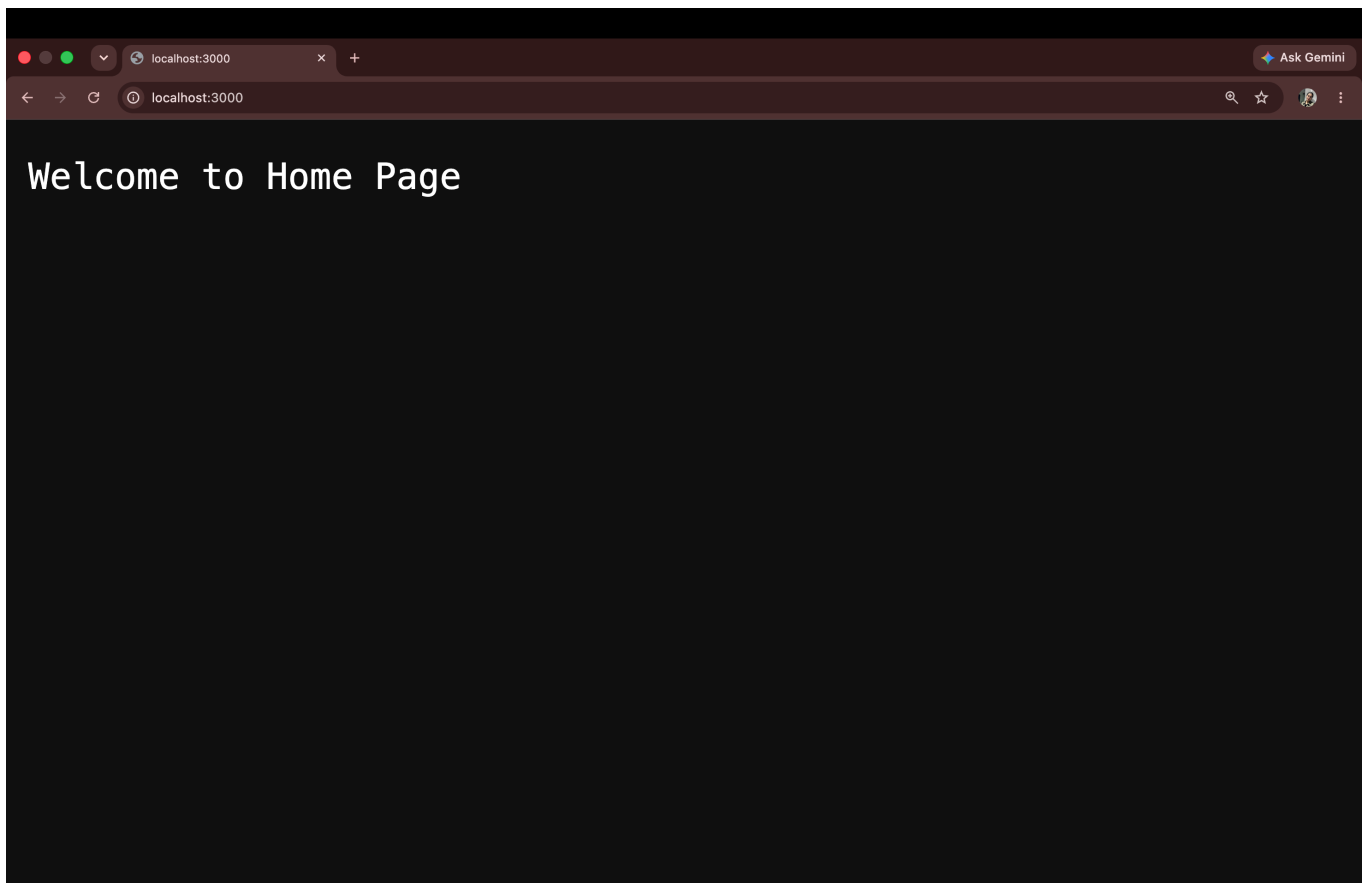
Route 1:- Home Page

URL :-

<http://localhost:3000/>

Output :-

Welcome to Home Page



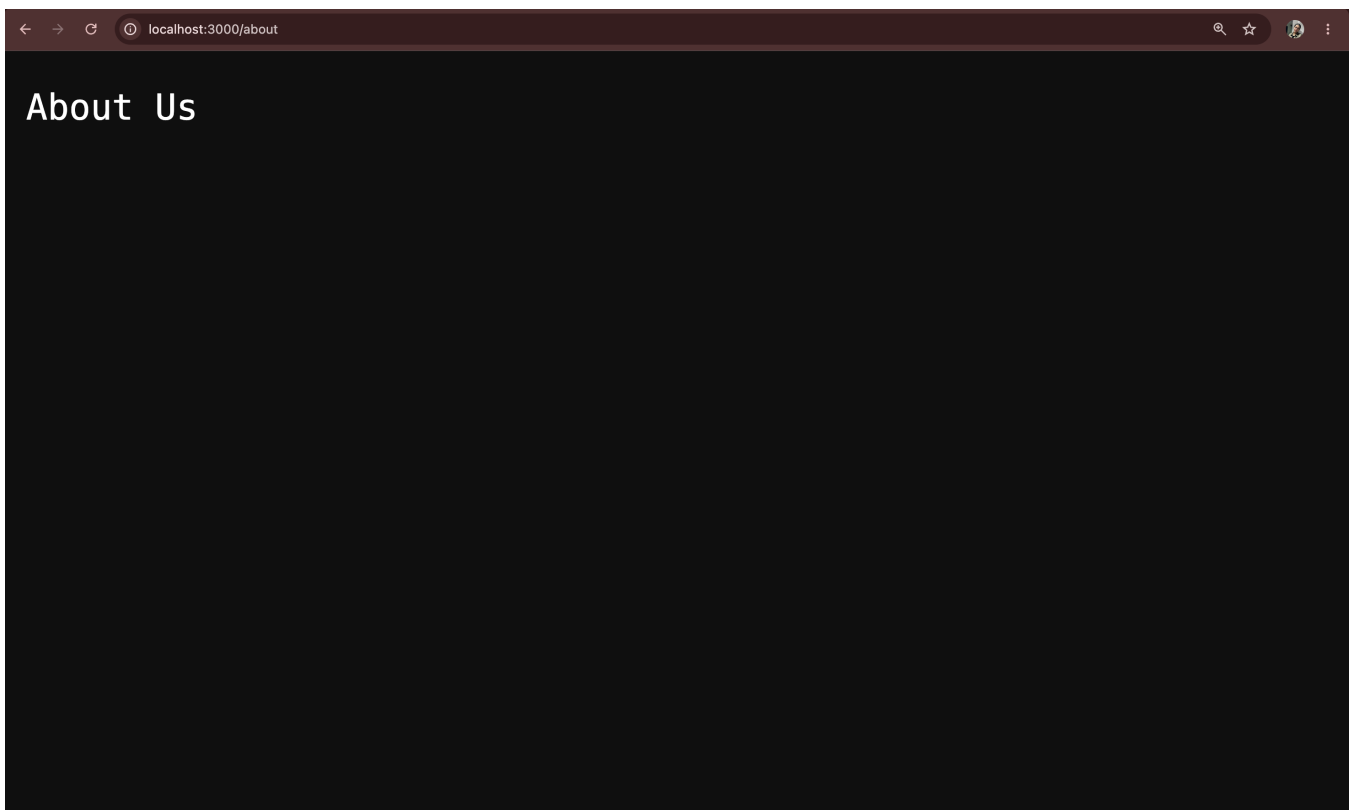
Route 2 :- About Page

URL :-

<http://localhost:3000/about>

Output :-

About Us



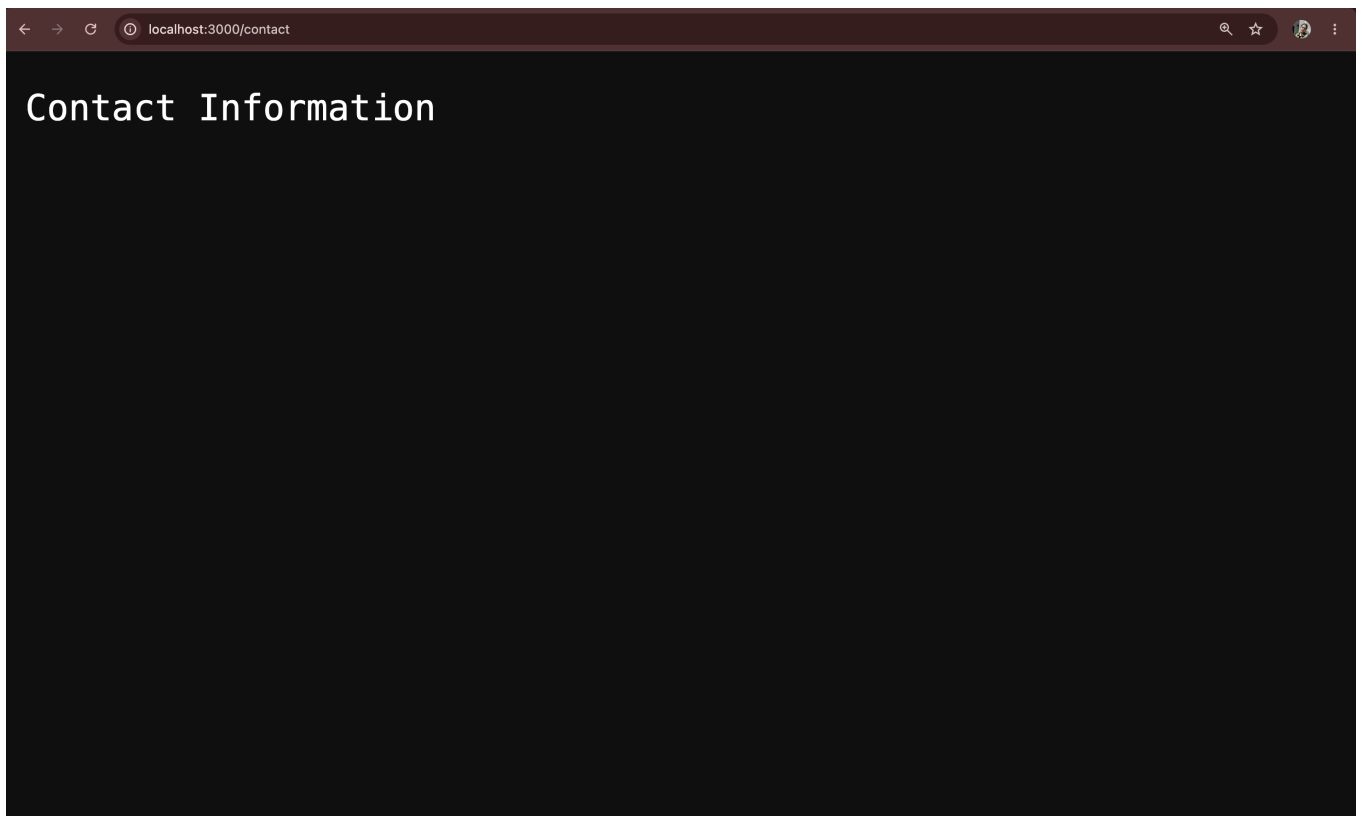
Route 3 :- Contact Page

URL :-

<http://localhost:3000/contact>

Output :-

Contact Information



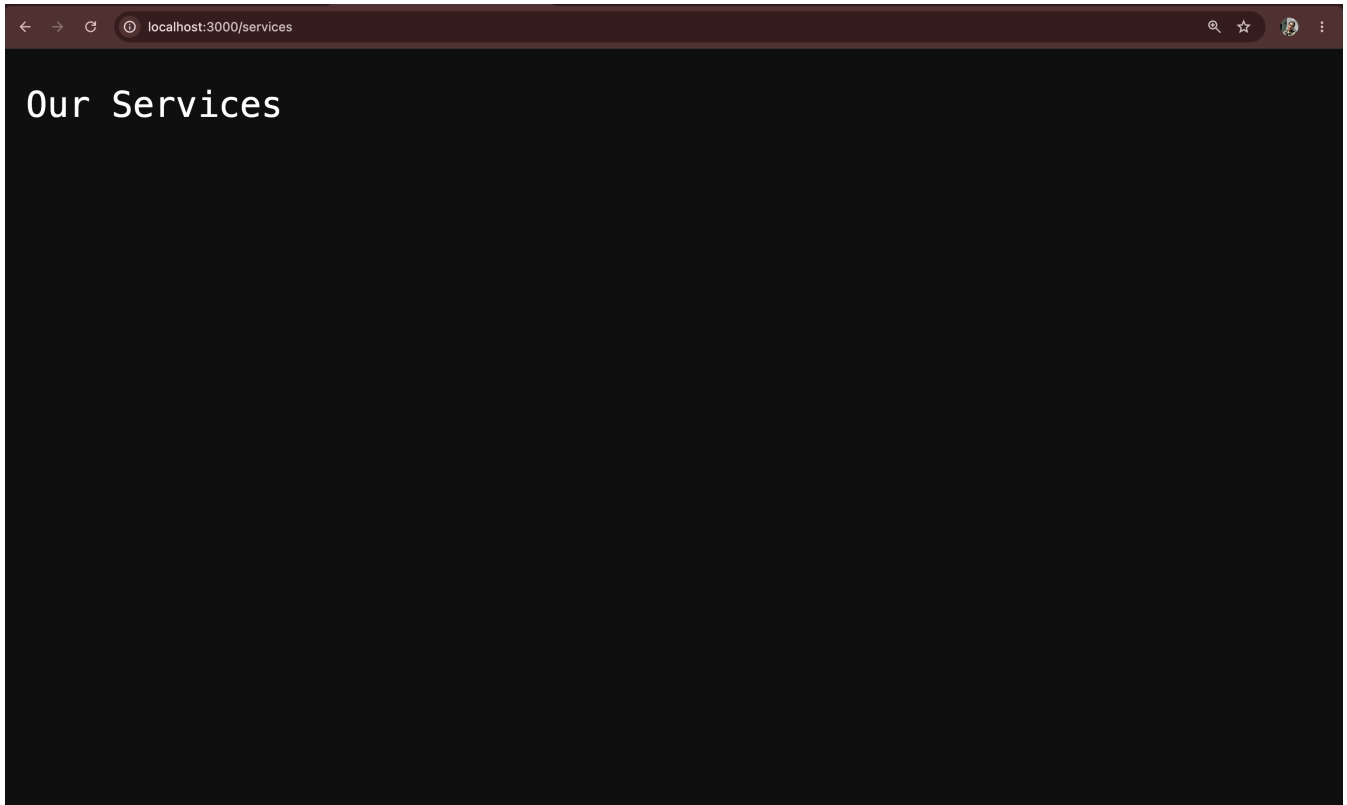
Route 4 :- Services Page

URL :-

<http://localhost:3000/services>

Output :-

Our Services



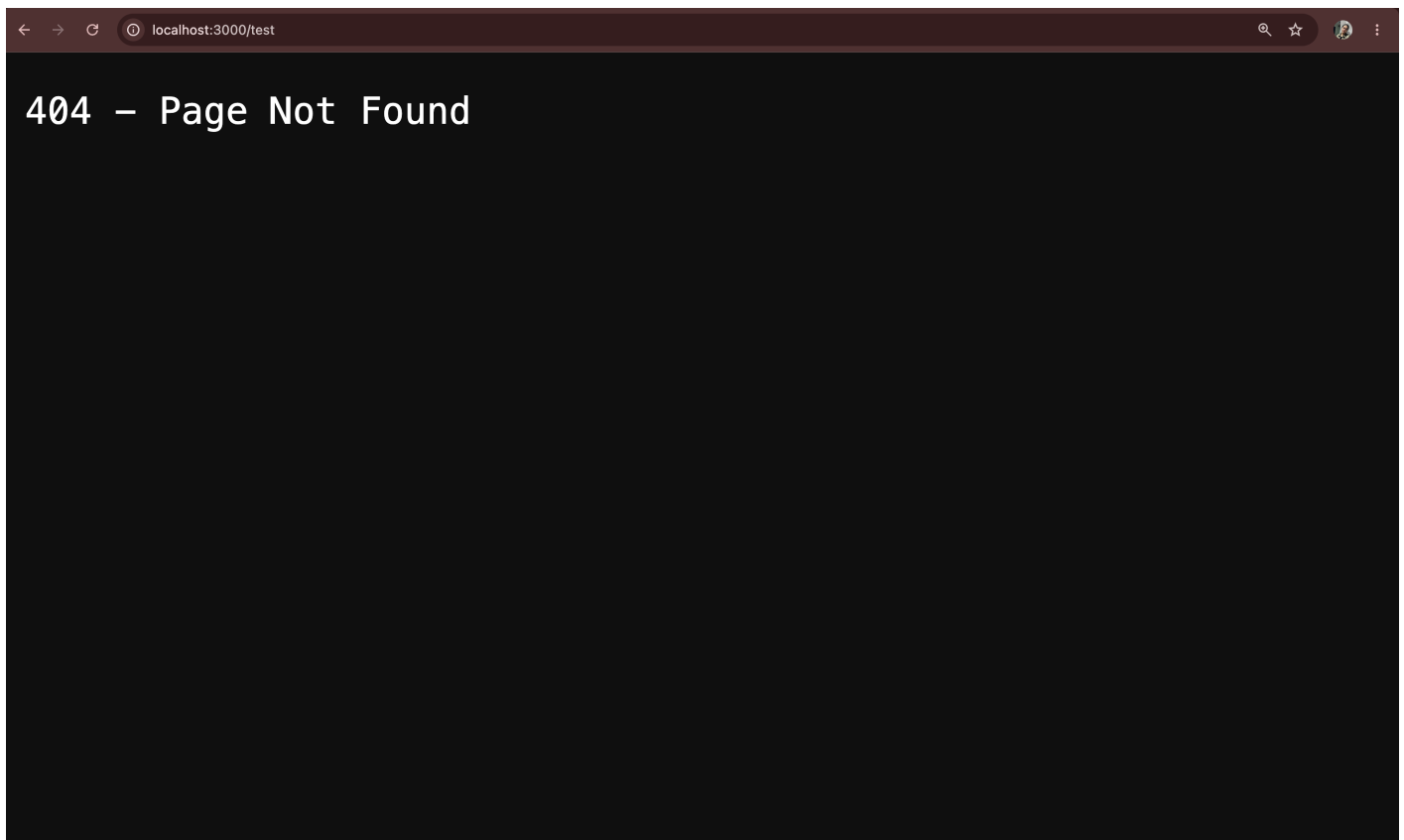
Route 5 :- Invalid Route

URL :-

<http://localhost:3000/test>

Output :-

404 - Page Not Found



5. Concepts Covered

1. Routing

Routing is used to provide different responses for different URLs.

For example :-

/	→ Welcome to Home Page
/about	→ About Us
/contact	→ Contact Information
/services	→ Our Services

2. URL Handling

The server checks the requested URL using :-

```
req.url
```

For example :-

```
req.url === '/about'
```

checks whether the browser requested the /about route.

3. HTTP Status Codes

The server uses HTTP status codes to indicate the result of a request.

For successful routes :-

```
res.writeHead(200, ...)
```

200 means the request was successful.

For an invalid route :-

```
res.writeHead(404, ...)
```

404 means the requested page was not found.

Name :- Rashmeet Kaur Nanade
Cohort :- Demis Hassabis
Roll No :- 150096725165
Subject :- Backend Development –
Node.js, MongoDB & Firebase.
Course :- B.Tech Computer Science &
Engineering
Professor :- Ricky Kumar

ASSIGNMENT - 5

PERSONAL PORTFOLIO SERVER

```
JS portfolioServer.js > server > http.createServer() callback
1  const http = require('http');
2
3  const server = http.createServer((req, res) => {
4
5    if (req.url === '/') {
6
7      res.writeHead(200, { 'Content-Type': 'text/html' });
8
9      res.end(`
10     <html>
11       <head>
12         <title>Home - Personal Portfolio</title>
13       </head>
14
15       <body>
16         <h1>Welcome to My Portfolio</h1>
17
18         <p>Hello! Welcome to my personal portfolio.</p>
19
20         <nav>
21           <a href="/">Home</a> |
22           <a href="/about">About Me</a> |
23           <a href="/skills">Skills</a> |
24           <a href="/projects">Projects</a> |
25           <a href="/contact">Contact</a>
26         </nav>
27
28         <h2>Home</h2>
29
30         <p>
31           This portfolio showcases my skills, projects,
32           and technical knowledge.
33         </p>
34       </body>
35     </html>
36   `);
```

```

37
38 }
39
40 else if (req.url === '/about') {
41
42     res.writeHead(200, { 'Content-Type': 'text/html' });
43
44     res.end(`
45         <html>
46             <head>
47                 <title>About Me</title>
48             </head>
49
50             <body>
51                 <h1>About Me</h1>
52
53                 <nav>
54                     <a href="/">Home</a> |
55                     <a href="/about">About Me</a> |
56                     <a href="/skills">Skills</a> |
57                     <a href="/projects">Projects</a> |
58                     <a href="/contact">Contact</a>
59                 </nav>
60
61                 <h2>About Me</h2>
62
63                 <p>
64                     I am a Computer Science student interested in
65                     web development, programming, and backend development.
66                 </p>
67
68                 <p>
69                     I am continuously learning new technologies and
70                     developing projects to improve my technical skills.
71                 </p>
72             </body>
73         </html>
74     `);
75
76 }
77
78 else if (req.url === '/skills') {
79
80     res.writeHead(200, { 'Content-Type': 'text/html' });
81
82     res.end(`
83         <html>
84             <head>
85                 <title>Skills</title>
86             </head>
87
88             <body>
89                 <h1>My Skills</h1>
90
91                 <nav>
92                     <a href="/">Home</a> |
93                     <a href="/about">About Me</a> |
94                     <a href="/skills">Skills</a> |
95                     <a href="/projects">Projects</a> |
96                     <a href="/contact">Contact</a>
97                 </nav>
98
99                 <h2>Technical Skills</h2>

```



```

101         <ul>
102             <li>C</li>
103             <li>C++</li>
104             <li>Python</li>
105             <li>HTML</li>
106             <li>CSS</li>
107             <li>JavaScript</li>
108             <li>React JS</li>
109             <li>Node.js</li>
110             <li>Git and GitHub</li>
111         </ul>
112     </body>
113 </html>
114 `);
115
116 }
117
118 else if (req.url === '/projects') {
119
120     res.writeHead(200, { 'Content-Type': 'text/html' });
121
122     res.end(`
123         <html>
124             <head>
125                 <title>Projects</title>
126             </head>
127
128             <body>
129                 <h1>My Projects</h1>
130
131                 <nav>
132                     <a href="/">Home</a> |
133                     <a href="/about">About Me</a> |
134                     <a href="/skills">Skills</a> |
135                     <a href="/projects">Projects</a> |
136                     <a href="/contact">Contact</a>
137                 </nav>
138
139                 <h2>Projects</h2>
140
141                 <ul>
142                     <li>Musical Staff Sight-Reading & Ear-Training Workspace</li>
143                     <li>Geography Map Quiz Game</li>
144                     <li>FAQify - React FAQ Accordion</li>
145                     <li>Personal Portfolio Website</li>
146                 </ul>
147
148                 <p>
149                     These projects helped me improve my programming,
150                     frontend development, and problem-solving skills.
151                 </p>
152             </body>
153         </html>
154     `);
155
156 }
157

```

```
158 else if (req.url === '/contact') {
159
160     res.writeHead(200, { 'Content-Type': 'text/html' });
161
162     res.end(`
163         <html>
164             <head>
165                 <title>Contact</title>
166             </head>
167
168             <body>
169                 <h1>Contact Details</h1>
170
171                 <nav>
172                     <a href="/">Home</a> |
173                     <a href="/about">About Me</a> |
174                     <a href="/skills">Skills</a> |
175                     <a href="/projects">Projects</a> |
176                     <a href="/contact">Contact</a>
177                 </nav>
178
179                 <h2>Get in Touch</h2>
180
181                 <p>Email:- krashmeet033@gmail.com</p>
182
183                 <p>
184                     You can contact me regarding my projects,
185                     technical skills, and learning journey.
186                 </p>
187             </body>
188         </html>
189     `);
190 }
```

```

191     }
192
193     else {
194
195         res.writeHead(404, { 'Content-Type': 'text/html' });
196
197         res.end(`
198             <html>
199             <head>
200                 <title>404 - Page Not Found</title>
201             </head>
202
203             <body>
204                 <h1>404 - Page Not Found</h1>
205
206                 <p>The page you are looking for does not exist.</p>
207
208                 <a href="/">Go Back to Home</a>
209             </body>
210             </html>
211         `);
212     }
213 }
214
215 });
216
217 server.listen(3000, () => {
218     console.log('Server is running on http://localhost:3000');
219 });

```

SOURCE CODE

Step 1 — Import HTTP module

```
const http = require('http');
```

This imports Node.js's built-in HTTP module.

We use it to create a web server.

Step 2 — Create the server

```
const server = http.createServer((req, res) => {
```

`http.createServer()` creates the server.

It receives two important objects :-

- `req` → request from the browser
- `res` → response sent back to the browser

Step 3 — Set HTML response

```
res.writeHead(200, { 'Content-Type': 'text/html' });
```

This tells the browser :-

- 200 → request was successful
- Content-Type: text/html → the response contains HTML

Step 4 — Create Navigation

Const navigation =`

```
<nav>
```

```
<a href="/">Home</a> |
```

```
  <a href="/about">About Me</a> |
```

```
  <a href="/skills">Skills</a> |
```

```
  <a href="/projects">Projects</a> |
```

```
  <a href="/contact">Contact</a>
```

```
</nav>
```

```
`;
```

This creates navigation links.

For example :-

```
<a href="/about">About Me</a>
```

When the user clicks About Me, the browser requests :-

```
/about
```

The server then handles that route.

Step 5 — Home Route

```
if (req.url === '/') {
```

This checks whether the requested URL is /.

If yes :-

```
res.end(  
  <h1>Welcome to My Portfolio</h1>  
  ...  
);
```

The Home page is displayed.

Browser URL :-

<http://localhost:3000/>

Step 6 — About Route

```
else if (req.url === '/about') {
```

This handles :-

<http://localhost:3000/about>

It displays :-

About Me

along with relevant information.

Step 7 — Skills Route

```
else if (req.url === '/skills') {
```

This handles :-

```
http://localhost:3000/skills
```

The skills are displayed using an HTML unordered list :-

```
<ul>
  <li>C</li>
  <li>C++</li>
  ...
</ul>
```

Step 8 — Projects Route

```
else if (req.url === '/projects') {
```

This handles :-

```
http://localhost:3000/projects
```

It displays the projects using :-

```
<ul>
  <li>...</li>
</ul>
```

Step 9 — Contact Route

```
else if (req.url === '/contact') {
```

This handles :-

```
http://localhost:3000/contact
```

It displays the contact details.

Step 10 — Invalid Route

```
else {
```

If none of the above routes match, the server executes this block.

For example :-

```
http://localhost:3000/test
```

There is no /test route, so the server sends :-

```
res.writeHead(404, { 'Content-Type': 'text/html' });
```

Here :-

```
404
```

means

Page Not Found

Then :-


```
res.end(  
  <h1>404 - Page Not Found</h1>  
  ...  
);
```

displays the error page.

Step 11 — Start the Server

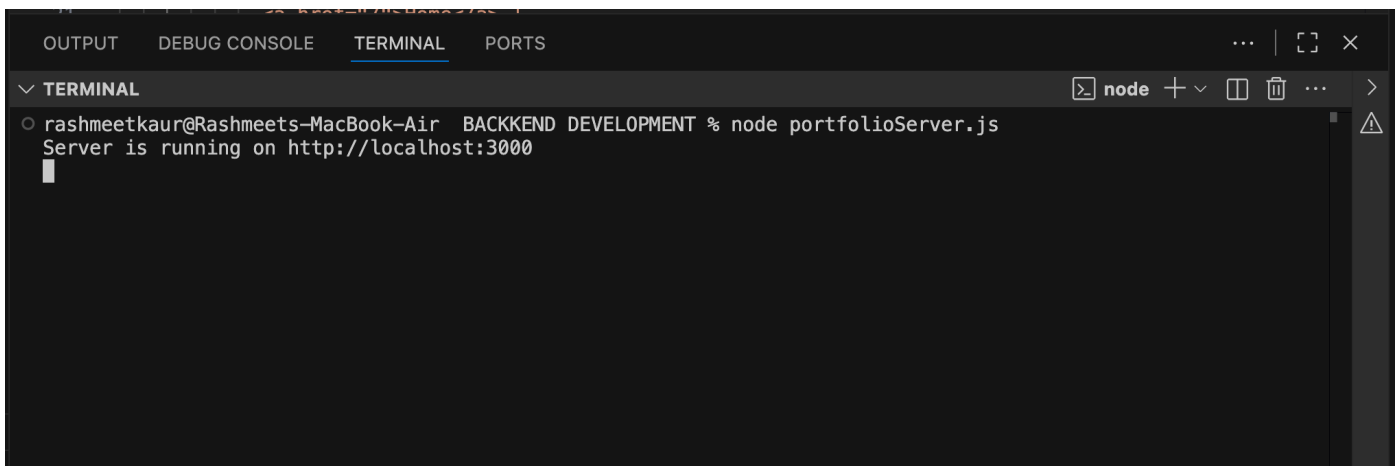
```
server.listen(3000, () => {  
  console.log('Server running at http://localhost:3000');  
});
```

This starts the server on :-

So your website is available at :-

<http://localhost:3000>

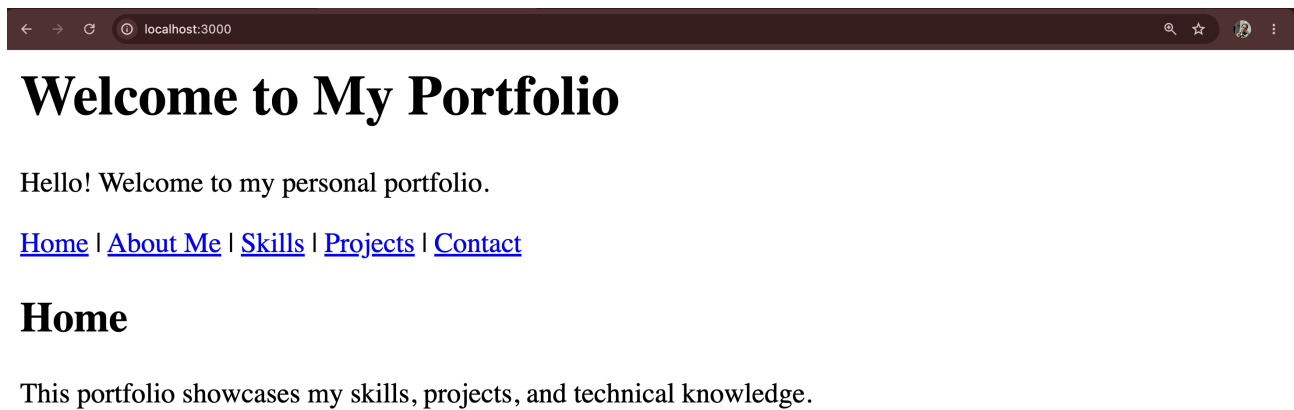
3. Terminal Output



4. Browser Outputs

4. Browser Outputs

Output 1 — Home



Output 2 — About Me



About Me

[Home](#) | [About Me](#) | [Skills](#) | [Projects](#) | [Contact](#)

About Me

I am a Computer Science student interested in web development, programming, and backend development.

I am continuously learning new technologies and developing projects to improve my technical skills.

Output 3 — Skills



My Skills

[Home](#) | [About Me](#) | [Skills](#) | [Projects](#) | [Contact](#)

Technical Skills

- C
- C++
- Python
- HTML
- CSS
- JavaScript
- React JS
- Node.js
- Git and GitHub

Output 4 — Projects



My Projects

[Home](#) | [About Me](#) | [Skills](#) | [Projects](#) | [Contact](#)

Projects

- Musical Staff Sight-Reading & Ear-Training Workspace
- Geography Map Quiz Game
- FAQify - React FAQ Accordion
- Personal Portfolio Website

These projects helped me improve my programming, frontend development, and problem-solving skills.

Output 5 — Contact



Contact Details

[Home](#) | [About Me](#) | [Skills](#) | [Projects](#) | [Contact](#)

Get in Touch

Email:- krashmeet033@gmail.com

You can contact me regarding my projects, technical skills, and learning journey.

Output 6 — Invalid Route / 404



404 - Page Not Found

The page you are looking for does not exist.

[Go Back to Home](#)